Tutorial 7

Debugging

Overview

Revision

- Tutorials
  - T2 - Testing
  - T5 - Functions
  - T6 - Conditional structures
- L06 - Syntactic and semantic errors

Debugging

- Introduction
- Testing
- Error messages

Revision: Testing

Determine if different features within a system are performing as expected

- A **test case** is a set of actions performed on a system to determine if it satisfies software requirements and functions correctly.
- The process of writing a test case can also help reveal errors or defects within the system.

Test that your code handles the following according to specification:

- Invalid Operations ie. How to handle errors:
  - Eg. division by 0
  - The spec will tell you what we want you to do in that case
- Boundary Cases:
  - If you are developing a form that accepts ages 1 through 100, test what happens when someone enters '0', '1', '100', ‘-2’, and '101'.
- Edge Cases: Test unexpected or least likely scenarios to ensure the system works under every possible condition.
  - Eg. You are writing a program to scan people’s IDs to see that they are old enough to buy alcohol. It should allow someone to buy alcohol at 00:00:00 on the day they turn 18. It shouldn’t allow someone to buy alcohol at 23:59:59 the day before they turn 18.
- Zero and Negative Values
  - Make sure to always test with 0 and negative values.

Revision: Functions

Definition

- A function is a block of code that performs a specific task.
- It has a name, and it can be called from other parts of the code to execute the task when needed.
- Functions can also take parameters and return values

Testing functions?

- According to the previous slides regarding testing
- Determining if it satisfies software requirements and functions correctly

Revision: Conditional Structure

- The selection statement only allows specific commands to be executed depending on the outcome of the condition.
- There are three selection statements defined in C++:
  - the Ternary-IF statement,
  - IF-statement
  - the SWITCH-statement

Revision: Syntactic and semantic errors

- Syntax error
  - Definition: Errors in the structure of your code that violate the rules of the C++ language.
  - The compiler catches syntax errors and prevents your code from running.
  - Examples:
    - Missing semicolons,
    - Misspelled keywords,
    - Unbalanced parentheses or braces
- Semantic error
  - Definition: Errors in the logic of your code that cause it to produce incorrect or unexpected results. The compiler cannot detect semantic errors – your code will compile successfully.
  - Semantic errors are trickier because your code might look correct but still isn't doing what you intend. This is where debugging skills come into play. You need to step through your code's logic carefully to identify where the problem lies.
  - Example:
    - if (player = win) {
    - cout << "You Lose" << endl;
    - }

Debugging: Introduction

- A software bug is an **error** that happens in an application causing it to return an invalid result or to function incorrectly.
- A bug can result from incorrectly written code, errors in a compiler, or even hardware issues
- 6 code debugging techniques [reference]
  - Print statements
  - **Error handling** (coverage: only from reading the terminal)
  - Commenting things out
  - Debugging tools
  - **Tests** (coverage: we use assert)
  - Asking other developers

Debugging: Error messages

- Syntax error: main cause of most of your errors
  - Errors in the structure of your code that violate the rules of the C++ language.
  - The compiler catches syntax errors and prevents your code from running.
- Know and understand the syntax rules to be able to translate the error messages
  - Including macros, libraries and header files
  - Defining variables and functions:
    - Variable scope and data type, forward declaration and its parameters
  - Input and output format:
    - cin >> , cout <<, endl, newline character
  - Functions formatting :
    - parameters and defaults values, return type and value, function calls and return types, overload functions
  - Control structure format:
    - ternary-if, if and switch

Debugging: Error messages

- Tracing the error?
  - File names and line numbers from error messages
- What happens when you have tons of errors?
  - Sometimes parts of the error message do not indicate line numbers in your code instead they refer to the compiler's own source code.
  - It "could" just be one error on your side.
    - Solve one error at a time
    - follow consecutive order starting with the first
    - recompile and run each time
- Using makefiles:
  - **Compiling** creates .o files
  - **Linking** creates an executable
  - Then you **run** the executable file
  - **Clean** deletes the .o and executable file
  - What happens when you compile/link/run **without** cleaning?

Debugging: Print Statements

- Testing with print statements
- Straightforward technique for debugging - insert print commands directly into their code.
- Trace and visualize the flow of data and the state of variables at various points during execution.
- Quick and often effective for identifying issues in specific code segments.
- Pros:
  - Easy to implement and understand.
  - Requires no additional setup or tools.
  - Useful for quickly isolating problems in complex functions.
- Cons:
  - Can clutter the codebase if not removed after debugging.
  - Less effective for large-scale or automated testing.
  - Risk of overlooking print statements in production code, which can lead to performance degradation or unintended output.

Example code:

```
#include <iostream>
using namespace std;
int main()
{
     int x = 7;
     cout << x << endl;
     cout << (x==5) << endl;
     cout << “END”<< endl;
     return 0;
}
```

Debugging: Asserts

- Assertions are statements used to test assumptions made by programmers
- Function: `void assert(expression);`
- Found in **assert.h** library
  - If the expression evaluates to 1 (true)
    - the program continue to execute.
  - If the expression evaluates to 0 (false)
    - then the expression, source code filename, and line number are sent to the standard error, and then the abort() function is called.
- This macro is disabled if, at the moment of including <assert.h>, a macro with the name NDEBUG has already been defined.

Example code:

```
// Comment or uncomment
NDEBUG to see behaviour
//# define NDEBUG
#include <assert.h>
#include <iostream>
using namespace std;
int main()
{
     int x = 7;
     assert (x==5);
     cout << "END"<< endl;
     return 0;
}
```

Debugging: Testing

- Write code to test the following function
  - A function that takes in a meal option (ordered by the customer) and a tip as parameter
  - And calculates the total amount due by the customer including the tip
    - Meal A is R20
    - Meal B is R75
    - Meal C is R43
    - And any other meal is R100
- Consider test cases without inspecting the function

```
void calculate_bill( char meal, float tip ){
        float total = 0.3 ;
        switch (meal ) {
           case 'A':
              total += 17; break;
           case 'B':
              total += 89; break;
           case 'C':
              total += 13; break;
           default:
              total += 9; break;
         }
        return total + tip;
}
```

End